PROJECT REPORT · STATIC MALWARE ANALYSIS

Hierarchical CFG-GNN for malware behaviour localisation

Function-level detection of anti-debug, keylogger, and injection behaviour in 32-bit Windows PE files, followed by basic-block localisation inside the functions selected by Stage 1.

Report date
23 October 2026

Scope
BODMAS v2 clean x86 PE corpus

Status
Three-target pipeline + single-binary inference

0.4890	0.4586	0.3723	116
STAGE 1 TEST MACRO F1	STAGE 2 ORACLE BLOCK F1	STAGE 2 END-TO-END F1	FUNCTIONS IN DEMO BINARY

Executive summary

The project now has a complete inference path: one x86 PE goes through angr CFG recovery, the frozen 187-feature block representation, a function-level GCN and a per-block head, and comes out as a JSON report plus a sample-specific PDF with function scores and basic-block addresses. The demo binary is a held-out test sample; the CLI selected one anti-debug function and 21 injection functions, and localised 205 injection blocks. The anti-debug function was selected at the function level but no block crossed the conservative 0.94 Stage 2 threshold, so its block list is empty by design.

> **What is measured.** Stage 1 is evaluated on functions, with per-target precision/recall/F1 and average precision. Stage 2 is evaluated on blocks with precision/recall/F1 and IoU, both on oracle positive functions and end-to-end with Stage 1 selection. Test was evaluated once; thresholds were selected on validation only.

Key findings

- **Injection is the strongest target end-to-end.** Stage 1 test F1 is 0.4774; Stage 2 end-to-end block F1 is 0.5202 with recall 0.9470. This is consistent with injection's larger positive support and API-call seed structure.
- **Keylogger is the weakest at function level.** Test F1 is 0.3376. The label is concentrated: 75.7% of positives come from one cluster and the feature is largely keyboard-state API presence.

- **Anti-debug function detection is moderate, block localisation is conservative.** Function F1 is 0.6522 on 27 test functions. The block head reaches precision 1.0000 but recall 0.0588 on oracle functions; the selected threshold is too strict for reliable block recovery.
- **Features are identical between corpus and single-binary inference.** A parity test compares all 36,649 block rows for the demo sample and obtains exact equality for features, addresses, function membership, edges, and model probabilities.
- **No malware-family attribution is produced.** "Not selected" means no target rule was selected; it is not a benign verdict. The model is trained on malware only.

Metrics in this report are the committed held-out test metrics. The demo sample and its graph are git-ignored; the report contains addresses and scores, not sample bytes.

1. Task and data contract

Stage 1 predicts three independent behaviours for every recovered function. The labels are CAPA namespace matches, not hand-verified ground truth. A function may match more than one target. Class 0 is "no target selected", not "benign".

target	CAPA namespace	meaning
`anti_debug`	`anti-analysis/anti-debugging`	anti-debugging / anti-analysis rules
`keylogger`	`collection/keylog`	keystroke-collection rules
`injection`	`host-interaction/process/inject` , `load-code/pe/inject-`	process / PE injection rules
background	none of the above	not selected; **not benign**

Corpus and labels

- 2,733 clean samples in the frozen manifest; 2,729 graphs recovered (4 angr failures).
- 2,749,125 function rows in the three-target split.
- Positive functions: **448 anti-debug**, **1,255 keylogger**, **1,349 injection**.
- Split protocol: `three-target-cluster-stratified-v3` , 70/15/15 by cluster. The old v2 `main.json` is not used for these targets.
- Test supports: 27 anti-debug, 153 keylogger, 202 injection functions.

Stage 2 supervision

- 120,319 masked blocks in 3,044 positive functions.
- Positive blocks: anti-debug 1,432, keylogger 9,504, injection 15,263.
- Labels combine CAPA evidence locations with import-resolved API-call seeds. API seeds dominate keylogger and injection positives.
- Test blocks: 136 anti-debug, 1,344 keylogger, 2,094 injection.

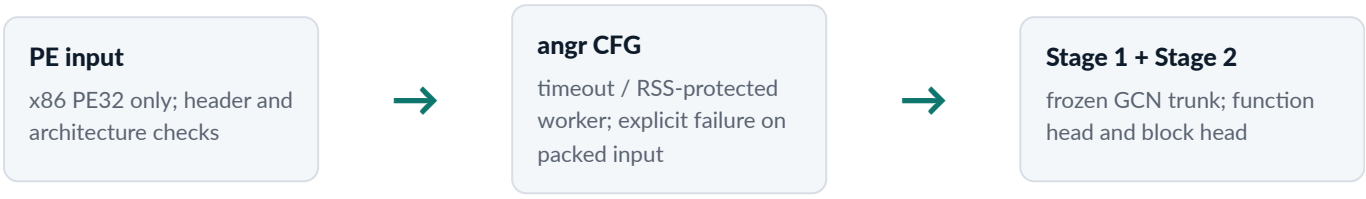
187-dimensional block feature

dims	feature
0–114	114 opcode frequencies + `other`
115–119	log block size, log instructions, log in-degree, log out-degree, is-entry
120–183	64 hashed API-reference buckets

dims	feature
184–186	log resolved API calls, log distinct buckets, has API call

API references are resolved through PE import thunks and `call/jmp dword ptr [IAT]` sites. No CAPA rule name or label value is an input feature. Features are intraprocedural only.

Pipeline



The inference CLI calls the same feature builder and the same model classes as training. It is not a separate reimplementation.

2. Stage 1: function classification

The GCN has two message-passing layers, a residual skip, mean+max pooling, and a small MLP head with three sigmoid outputs. Edges are intraprocedural. Model selection used validation only; the test split was evaluated once.

target	support	precision	recall	F1	AP
`anti_debug`	27	0.7895	0.5556	**0.6522**	0.4836
`keylogger`	153	0.3292	0.3464	**0.3376**	0.2477
`injection`	202	0.3761	0.6535	**0.4774**	0.3327
macro				**0.4890**	

> **Any-target selection**
> Precision 0.4355, recall 0.5729, F1 0.4948.

> **Background / not selected**
> Precision 0.9982, recall 0.9969, F1 0.9976.

How to read this table

- The macro F1 is **0.4890**. It is not a family-wide generalisation claim: the test supports are small and clustered.
- Injection has the largest test support (202) and the highest F1 (0.4774). Keylogger has 153 positives but only 0.3376 F1, reflecting the weak and concentrated label.
- Anti-debug has only 27 test positives. Its F1 (0.6522) has a wide uncertainty even though precision is relatively high (0.7895).

> **Label quality caveat.** CAPA namespaces are observational labels. 75.7% of keylogger positives and 61.2% of anti-debug positives come from one cluster each; injection is 32.9%. The audit is committed under `results/audit/`. The labels are reported as-is; they are not renamed or dropped.

3. Stage 2: block localisation

The Stage 2 trunk is the frozen Stage 1 GCN; only a per-block head is trained. “Oracle” uses all true positive functions, so it isolates block-head quality. “End-to-end” uses only functions selected by Stage 1, so it includes Stage 1 false positives and missed functions.

target	support blocks	oracle P	oracle R	oracle F1	oracle IoU	e2e P	e2e R	e2e F1	e2e IoU
`anti_debug`	136	1.0000	0.0588	**0.1111**	0.0588	0.6667	0.1081	**0.1860**	0.1026
`keylogger`	1344	0.4312	0.5900	**0.4983**	0.3318	0.2980	0.6598	**0.4106**	0.2583
`injection`	2094	0.6462	0.9413	**0.7663**	0.6212	0.3585	0.9470	**0.5202**	0.3515
macro				**0.4586**	**0.3373**			**0.3723**	**0.2375**

Observations

- Macro oracle F1 is **0.4586** (IoU 0.3373); macro end-to-end F1 is **0.3723** (IoU 0.2375).
- Injection dominates: oracle F1 0.7663 and end-to-end F1 0.5202, with end-to-end recall 0.9470.
- Keylogger oracle F1 is 0.4983; end-to-end F1 is 0.4106. The block labels are API-call-heavy, which the head can recover better than the function-level keylogger pattern.
- Anti-debug oracle precision is 1.0000 but recall is 0.0588 on 136 test blocks. The 0.94 threshold was selected on 27 validation functions and is too conservative. Its block AP is 0.336, so ranking is better than thresholded recall suggests.

> **Diagnostic, not ground truth.** Stage 2 labels come from CAPA evidence plus API-call seeds. The metrics measure recovery of those seeds, not every malicious block. A localised block is a lead for an analyst, not proof that the block executes a malicious action.

4. Single-binary inference

The implementation adds `src/infer/run.py` with a CLI, a reusable pipeline, and a JSON report builder. It validates the PE, optionally extracts the CFG with the existing worker, rebuilds the exact features, loads the frozen checkpoints, runs both stages, and writes an atomic JSON report plus a sample-specific PDF report. The PDF is generated by default next to the JSON; `--no-pdf` disables it.

```
./venv/bin/python -m src.infer.run \
  --binary <x86-pe> \
  --out results/infer/<sample>.json
# also writes results/infer/<sample>.pdf

# optional: skip extraction for an existing graph
./venv/bin/python -m src.infer.run \
  --binary <x86-pe> --graph <graph.npz> \
  --out /tmp/opencode/report.json
```

Demo run: held-out test sample

`25c5f0d873e8167464eab10187deb28da54656398d26c7dad9ff1918a25d3dd0` · 116 functions · 1,912 blocks · base `0x10000000` · API imports resolved: 119 · thunks: 388

target	status	flagged functions	localized blocks	max function	stage 1 thr.	stage 2 thr.
`anti_debug`	**PRESENT**	1 / 116	0	0.9989	0.9500	0.9400
`keylogger`	absent	0 / 116	0	0.9227	0.9400	0.5000
`injection`	**PRESENT**	21 / 116	205	0.9996	0.9500	0.6400

> **Anti-debug has a function flag but no block flag.** function `0x3e20` is selected at 0.9989, but its best block score is 0.8085 < 0.94; no block is reported. This is the expected consequence of the conservative Stage 2 anti-debug threshold, not an extraction or CLI failure.

Top injection functions and first localised block

function RVA	function score	max block	# blocks	top block RVA	top block score
`0x5880`	0.9996	0.9974	12	`0x5afe`	0.9974
`0x6a40`	0.9995	0.9974	13	`0x6aca`	0.9974
`0x7330`	0.9995	0.9964	28	`0x779d`	0.9964
`0x6f40`	0.9993	0.9964	5	`0x711e`	0.9964
`0x2ad0`	0.9976	0.9992	7	`0x2c45`	0.9992

Static decode spot-check

Capstone decoded the first six localised injection blocks from the saved PE. This checks address mapping and shows the IAT calls behind the Stage 2 API seeds; it is not a Ghidra/manual label verdict.

function RVA	block RVA	IAT call	bytes decoded	instructions
`0x5880`	`0x5afe`	`KERNEL32.dll!lstrlenW`	7 / 7	`push edi; call dword ptr [0x1000a0b0]`
`0x5880`	`0x5ad4`	`KERNEL32.dll!lstrlenA`	13 / 13	`push dword ptr [esp + 0x14]; lea esi, [edi + eax*2]; call dword ptr [0x1000a160]`
`0x6a40`	`0x6aca`	`KERNEL32.dll!lstrlenW`	7 / 7	`push esi; call dword ptr [0x1000a0b0]`
`0x6a40`	`0x6b69`	`KERNEL32.dll!lstrlenA`	11 / 11	`push 0x1000feb0; call dword ptr [0x1000a160]`
`0x7330`	`0x779d`	`KERNEL32.dll!lstrlenW`	9 / 9	`push dword ptr [esi + 0x68]; call dword ptr [0x1000a0b0]`
`0x7330`	`0x76bc`	`KERNEL32.dll!GetWindowsDirectoryW`	14 / 14	`mov edi, eax; push 0x100; push edi; call dword ptr [0x1000a0f4]`

6/6 sampled blocks decoded completely; 6/6 end in an IAT call. The calls shown are consistent with the API-seed supervision, which is also why they are not independent proof of malicious semantics.

Feature parity
For the demo sample, single-graph and corpus features are bit-identical: 36,649 rows, 8,633 functions, 1,674 edges, 387 resolved API calls; maximum absolute feature difference 0.0. Stage 1 and Stage 2 probability differences are also 0.0. Two additional held-out samples reproduced the corpus batch selected sets exactly; see `results/infer/inference_validation.json`.

Architecture and failures
The CLI rejects known x64/ARM machines and PE32+ input before angr. Packed or malformed samples that cannot be recovered return exit code 2 with the worker error. They are never converted into a negative prediction.

Output contract

The JSON schema is `cfg-gnn.infer.v1`. Each target has `present`, `max_function_score`, `n_flagged_functions`, thresholds, and a function list. Each selected function has `func_va`, `func_rva`, `score`, `max_block_score`, and blocks sorted by Stage 2 score. The same data is rendered as a per-sample PDF by default; `--no-pdf` skips it. The complete field documentation is in `docs/inference.md`.

5. Limitations and honest interpretation

Labels and scope

- **Weak labels.** CAPA matches are not ground truth. The targets are namespace shorthand, not proof of intent.
- **Malware only.** There is no benign corpus. Background means “no target rule selected”, not “safe”.
- **Cluster concentration.** Keylogger and anti-debug results are dominated by one cluster each. Per-target scores should be read with cluster support, not as broad capability claims.
- **x86 PE only.** ELF, ARM, and x64 are out of scope; the 855 x64 BODMAS rows were packed and excluded by the v2 filter.

Pipeline and evaluation

- **Stage 2 is weakly supervised.** Its metrics measure recovery of CAPA/API seeds, not complete malicious-block annotation.
- **Anti-debug block threshold is unstable.** Only 27 validation functions; oracle F1 0.1111. A higher-recall threshold may be preferable for triage.
- **Static recovery limits.** Packed binaries, indirect jumps, and missed code reduce recall. angr failures are reported, not silently dropped.
- **Test reuse.** The test split has been evaluated once for each stage. Do not retune on it after design changes.

> **Open validation.** The Ghidra block-label spot-check remains open: 40 functions were selected and 0 verdicts are recorded. The inference CLI does not require Ghidra and does not claim that predicted blocks have been manually validated.

What the result does and does not claim

claim	status
The pipeline can score one x86 PE and return function flags plus block addresses.	**Implemented and run on a held-out sample**
Single-binary features and scores match the corpus path.	**Verified exactly on the demo sample**
The model detects all malicious functions in a binary.	**Not claimed; recall is limited, especially keylogger and anti-debug**
The model identifies every malicious basic block.	**Not claimed; Stage 2 labels are weak and anti-debug recall is low**
The model names a malware family.	**Not a target; no family classifier is trained or reported**
Predicted blocks are manually validated.	**Ghidra spot-check is still open**

Next engineering steps

1. Complete the Ghidra block-label spot-check on the 40 selected functions; record verdicts and disagreement reasons.
2. Compare the GCN encoder with a GAT encoder under the same split and frozen feature contract; this remains the planned modelling comparison.
3. Increase anti-debug and keylogger support or report per-cluster results with confidence intervals; do not retune on test.
4. Add a small analyst-facing wrapper around the JSON report (ranked functions, top blocks, and explicit extraction failures) once the output contract is stable.

Reproduction. Training and evaluation commands are in `README.md` and `docs/label_pipeline.md`; the inference contract is in `docs/inference.md`. The committed report generator is `scripts/make_final_report.py`.

Appendix: implementation and artifacts

Inference code

`src/infer/pipeline.py`
`src/infer/report.py`
`src/infer/pdf.py`
`src/infer/run.py`

Tests and docs

`tests/test_infer.py`
`docs/inference.md`
81 tests passing

Artifacts

`results/train/stage1/`
`results/train/stage2/`
`results/infer/`

Committed vs local-only

Source, tests, documentation, metrics, reports, and inference JSON are committed. Raw malware samples, CAPA JSON, graphs, corpus arrays, and model checkpoints are git-ignored. The report contains sample metadata and addresses, not sample bytes.

Checkpoint thresholds

stage	anti-debug	keylogger	injection
Stage 1	0.95	0.94	0.95
Stage 2	0.94	0.50	0.64

Test summary

The repository test suite covers CFG storage, label parsing and mapping, feature/corpus invariants, model batching, Stage 1/Stage 2 training helpers, the end-to-end evaluator, and the inference report/parity path. The final run reported **81 passing tests**.

Report generated by `scripts/make_final_report.py`. The PDF is printed from local HTML with Chromium; no external assets or network access are required.